Writing More Cypress Tests

Let's return to our Cypress tests. In the last chapter, we wrote a test that shows that our "add favorite" functionality works. Now we want to show that our "remove favorite" functionality also works.

We could just add more lines to the existing test to continue that scenario with a removal of favorite functionality. That's a reasonable thing to do in some circumstances. However, longer tests like that tend to be more brittle, and a failing test gives you less information because the failure might be anywhere along the sequence of the test.

But in order to get a test to a state where we can remove a favorite, we kind of have to go through all those same steps: log in a user and add a favorite. Ideally, we'd be able to start the test in that state. However, the cypress-rails gem is deliberately structured without a built-in mechanism to trigger test-specific Rails data setup from within a Cypress test.

Instead, what cypress-rails expects us to do is write a test-specific Rails controller action and invoke it from our tests using cy.request. Loading data from a controller has a couple of advantages. It clearly places the responsibility for setting up data in a regular Rails process that does regular Rails things and is therefore easier to understand. Also, it separates the test and the data in a way that if we were ever to need to either move Cypress to its own code repository or change the server from Rails, we could do so with minimal change to the test itself.

Here's what that looks like in code. We just need a controller that will provide a home for test setup. I've created a subdirectory for test controllers, which means that for Rails autoload purposes, I need to put the controller in a module:

chapter_15/01/app/controllers/test/setup_controller.rb

```
module Test
  class SetupController < ApplicationController
```